ANEXO EXAMEN DE CERTIFICACIÓN

Plan de Estudio	Desarrollo Aplicaciones FullStack Python
	Caso Biblioteca Vecinal

Caso “Biblioteca Vecinal (Fullstack Python)”

La **Unión de Juntas Vecinales de Panguipulli** ha decidido impulsar la lectura entre los miembros más jóvenes de su comunidad. Demostrados los enormes beneficios en comprensión lectora, razonamiento crítico y desarrollo lingüístico da el incentivo de la lectura temprana en los escolares es que la Unión ha debatido con frecuencia cómo poder ayudar en este aspecto.

Una de las propuestas que más apoyo concita es la de **Juan Lopez**, miembro de la unión, que se ha decidido por crear un sistema de bibliotecas vecinales. Cada biblioteca debe proveer de diversos

ejemplares de obras clásicas, y otras contemporáneas. Esto apoyado por un software desarrollado ad-hoc que permita gestionar los préstamos a los vecinos.

Luego de conocer los beneficios de desarrollar un sistema basado en el framework Django es que la Unión le ha solicitado a Ud. que desarrolle un MVP (DEMO) de una aplicación que gestione **una biblioteca**. De esta manera, y en conjunto con don Juan, podrán tomar la decisión de aprobar el desarrollo de un sistema más complejo, que permita solucionar los problemas de gestión y administración de las bibliotecas vecinales.

Por lo tanto usted será responsable de elaborar una DEMO de la plataforma, de modo de convencer a don Juan y al resto del equipo de contratar sus servicios como desarrollador(a) de software usando el stack Python/Django y el framework CSS Bootstrap. Desarrollando crear una aplicación Web escalable y robusta sin incurrir en excesivos tiempos de implementación.

Antecedentes

Luego de reunirse con la junta, actualmente se cuentan con los siguientes antecedentes:

- El sistema debe permitir a los vecinos **registrarse** y realizar **login** en el sistema.
- Además de los vecinos, debe existir un **usuario administrador**, que posee permisos distintos a los vecinos.
- Dentro de la plataforma, existen muchos libros, separados por **categoría** y por **tipo**.
- El **tipo** de un libro indica por cuántos días se presta el libro, y la multa por días de atraso.
- Un vecino debe poder ver la lista completa de libros, y debe poder filtrar por categorías.
- Un vecino debe poder arrendar un libro, y retornarlo. Cómo esto es una DEMO, al momento de arrendar un libro se debe especificar la fecha. Lo mismo que al retornarlo.
- No se puede retornar un libro con una fecha anterior al préstamo.
- Si un vecino devuelve el libro pasados los días permitidos, entonces se le debe informar de la multa calculada.
- Para esta DEMO considere que existe un único ejemplar de cada libro.
- Finalmente, el usuario administrador debe poder agregar libros en la plataforma.
- Para esta DEMO considere que las categorías y tipos los puede administrar con el **superuser** de Django-Admin. Pero este mismo usuario debe poder agregar libros de la plataforma.
- Como datos iniciales considere estas categorías:
    - Novelas
    - Autoayuda
    - Terror
- Y los siguientes tipos:
    - Estrenos (arriendo 4 días, multa por día de atraso: $2000)
    - Clásicos (arriendo 6 días, multa por día de atraso: $1500)
    - Estandar (arriendo 6 días, multa por día de atraso: $1000)

Requerimientos Funcionales del Sistema

Después de varias reuniones entre la junta y nuestro equipo técnico, los requerimientos que pudimos bajar son los siguientes:

- Desarrollar una aplicación Web basada en el prototipo propuesto en las ilustracione
- La aplicación se debe poder navegar desde la barra superior. En la misma debe saludarse al usuario por su nombre (si está logueado), y tener un botón y/o enlace para salir de sesión.
- Las pantallas accesibles si está logueado el usuario son:
    - Home (ruta: **/**)
    - Mis Arriendos (ruta: **/arriendos**)
    - Arrendar (ruta: **/libros/<id>/arrendar**)
    - Retornar (ruta: **/libros/<id>/retornar**)
    - Agregar Libros (ruta: **/libros** ; debe ser accesible sólo por el superuser).
- Las funcionalidades descritas deben estar protegidas para que sean realizadas sólo por usuarios registrados y autentificados correctamente. En el caso de que un usuario no esté registrado, el sistema debe redirigirlo a la pantalla para que haga Login o se Registre.
- En la vista principal (ruta: **/**) un vecino debe ver una tabla con todos los libros disponibles, con su respectiva categoría. También debe poder filtrar por categoría.
- En esta misma tabla debe dirigirse a la pantalla de arrendar.
- En la pantalla **Mis Arriendos** se deben poder redirigir a la pantalla **Retornar**.
- Al momento de retornar un libro, y en caso de que se justifique una multa, se debe comunicar al vecino cuánto debe pagar.
- Los formularios pueden validarse en el FrontEnd o en el BackEnd. Esto queda a su discreción
- Se puede implementar la autenticación de usuarios usando Django-Auth o un sistema customizado. Pero si se debe procurar que las contraseñas estén encriptadas.
- Las rutas deben ser exactas a lo que muestran las maquetas. La excepción de esto son las rutas de autenticación. En el caso de que use otras rutas se debe especificar en el **README.txt**.

Requerimiento Final

Cuando un usuario retorna un libro fuera de plazo se genera una multa. Agregue una pantalla, sólo accesible para el superuser, en donde se puedan ver todas las multas generadas por todos los vecinos, ordenadas por fecha en dirección ascendente. Considere un modelo para almacenar estas multas. Desarrolle este requisito sólo cuando el resto del sistema ya esté completado. No olvide commitear antes. ;)

Maquetas (Mockups)

Ilustración 1: Pantalla Principal

Ruta: /

Ilustración 2: Pantalla Agregar Libro (sólo accesible por Admin)

Ruta: /libros

Ilustración 3: Pantalla Mis Arriendos

Ruta: /arriendos

Ilustración 4: Pantalla para Arrendar

Ruta: /libros/<id>/arrendar

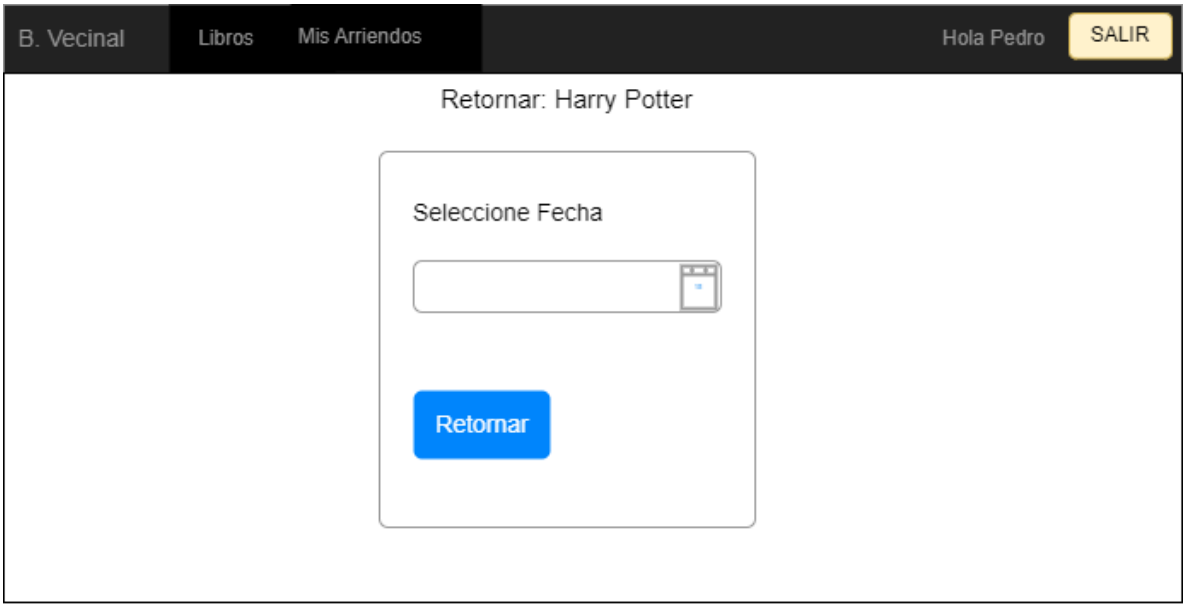


Ilustración 5: Pantalla para Retornar

Ruta: /libros/<id>/retornar

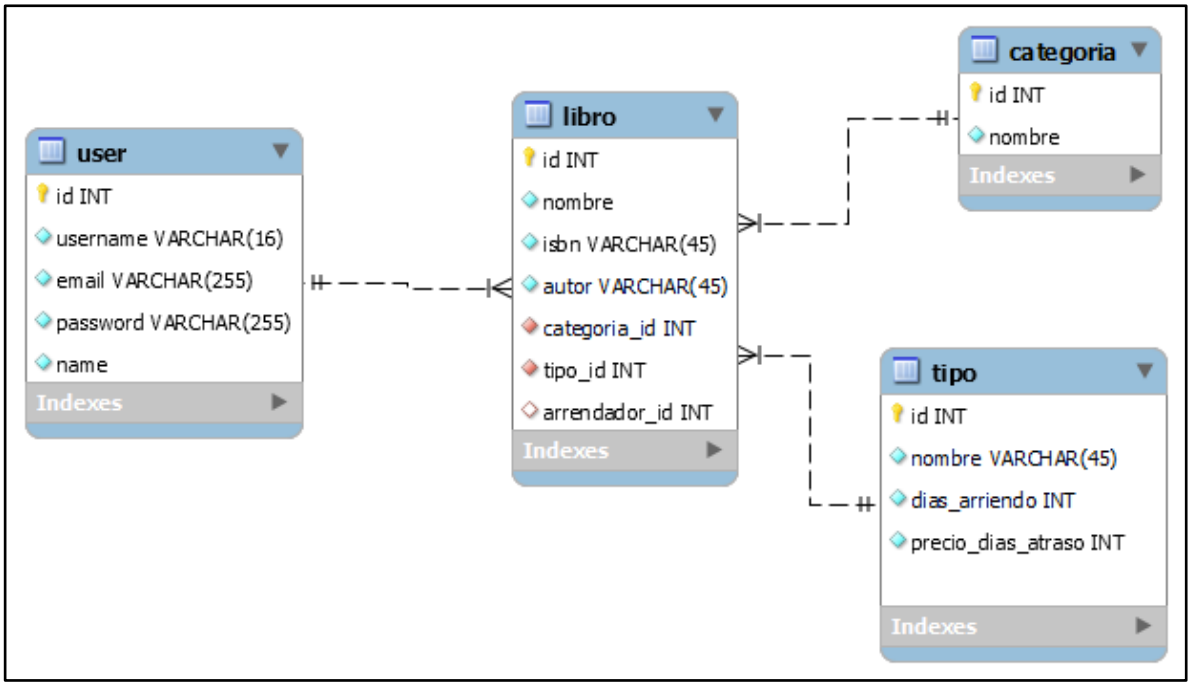
Requerimientos No Funcionales

El arquitecto del proyecto, le ha hecho las siguientes definiciones:

- El sistema debe utilizar una base de datos PostgreSql.
- El sistema debe construirse con Python/Django
- El sistema Web debe construirse bajo el patrón MVC.
- El acceso de usuarios se puede implementar con el modelo de Django Auth, o un modelo personalizado que usted prefiera.
- La capa de acceso a datos debe ser implementada con el ORM Django
- El archivo README.txt debe ir en la carpeta del proyecto.

Modelo de Datos

A continuación, se presenta el modelo de datos que ya ha sido diseñado por el equipo técnico del proyecto. Puede usarlo como guía para entender el problema, pero **no se preocupe** si los campos cambian de nombre al utilizar la Django ORM, o si desea quitar, agregar campos.



Entregable

Haga todos los supuestos que estime conveniente para la resolución del caso y déjelos debidamente comentados en el código fuente en caso de requerirse. Recuerde, al final del proyecto entregar lo siguiente:

- Código fuente del proyecto
- El proyecto debe llamarse exactamente **biblioteca_vecinal** (sin espacios ni mayúsculas, con guión bajo).
- Export de la base de datos con esquema y datos, (le puede consultar al instructor a cargo). Este modelo debe llamarse exactamente **biblioteca_vecinal.sql** (sin espacios ni mayúsculas, con guión bajo), y debe ir dentro de la carpeta del proyecto. Este export debe contener al menos 2 vecinos, 4 libros, 3 categorías, 3 tipos, y el superadmin.
- El username del superadmin de DjangoAdmin debe ser **admin**, y su contraseña debe ser **admin** (no otra contraseña).
- Archivo **requirements.txt** con las dependencias (puede usar también un **Pipfile**).
- Archivo **README.txt** con todas las indicaciones para ejecutar el proyecto. Debe contener las dependencias, y las credenciales para que los usuarios puedan acceder al sistema.

- Queda estrictamente prohibido compartir el código en github u otra plataforma.
- No se puede utilizar ningún framework Frontend reactivo ni Docker

DUMP

Para realizar un DUMP de la base de datos puede servirle este comando:

```
pg_dump -U postgres -h localhost nombre_db >> nombre_db.sql
```